



Overview document

Table des matières

Introduction

Équipe

Fiche ID

Pitch

Intentions

Concept de jeu

Conditions V/D

Déroulé d’une partie

3C

Meta-Boucle

Objectifs de jeu

Mécaniques

Déplacement

Bunny Hop

Riffle

Shotgun

Rocket

Bestiaires

Trash mob

Distance

Goliath

Level Design

UI

Armes

Points de vie

Références de Design

Gameplay

Visuelle

Sonore

Boucles d’objectifs

Court terme

Moyen terme

Long terme

Améliorations

Introduction

Ce document présente l’overview du projet de quatrième année de mastère du groupe SCAR pour l’ICAN
Le thème imposé par l’équipe pédagogique était un genre de jeu : Fast Arena Shooter



À partir de ce genre, nous avons élaboré des réflexions sur la notion des Fast Arena Shooter et des capacités déjà existantes dans certains jeux utilisant cette notion (Doom/Quake).
De cette idée, nous avons prototypé le concept de jeu présenté ici.

Équipe

Sarah EL MABSOUT
Game Artist

Charles Barrier
Game Designer

Antoine Dias
Programmeur

Raphaël ONNO
Game Designer

Fiche ID

Pitch

Prenez le contrôle d'un avatar aux capacités de défense multiple lié au déplacement, dans un environnement fermé incluant différents modules qui vont vous aider à venir à bout des vagues ennemies :

- Jeu 1 vs IA
- Fast Arena Shooter
- First Person Shooter
- Structure : Enchaînement de vague d'ennemis pendant une durée de 10 minutes
- Univers : Toits de building, arène close.
- PC
- Vise un public mid/hard core, cherchant du challenge dans les patterns du système

Intentions

Le projet RoofRush a pour objectif de proposer une expérience de gameplay mettant principalement les choix de déplacement du Joueur au premier plan.

Le Joueur devra utiliser et comprendre les différentes mécaniques proposés par le système, offense comme défensif, afin de venir à bouts des différents ennemis présent dans l'espace de jeu.

Nous voulions une tension constante sur le joueur pour les mettre en contrainte permanente face à toutes les situations de jeu qu'ils peuvent rencontrer.

De plus, le rechargerment des armes étant liés aux déplacements, ce qui aura pour conséquence d'obliger le joueur à prendre des risques tout en se m'étant en retrait, en utilisant la mécanique de bunny hop.

Concept de jeu

RoofRush est un jeu du genre Fast Arena Shooter, en vue FPS, qui se déroule dans un environnement 3D clos.

Le jeu fonctionne en enchaînement de vagues ennemis dans lesquelles le joueur doit faire face en lieu à des vagues successives d'ennemis dont leurs difficultés sont croissantes à chaque nouvelle vague pendant une durée de 10 minutes.

Pour s'en débarrasser, le joueur peut utiliser diverses armes dont leurs systèmes de rechargements est liés à des conditions de déplacements. Pendant une durée de 10 minutes le joueur devra faire face à de plus en plus d'ennemis à la difficulté croissante.

A la fin du temps impartie, le joueur doit détruire les ennemis encore présent, une fois qu'il n'y a plus d'ennemis, il peut avancer à la prochaine maps.

Conditions de Victoire

Victoire :

Si le joueur arrive à détruire toutes les vagues d'ennemis de son arène, il gagne.
Il débloquent l'accès à la prochaine map.

Défaite :

L'avatar dispose de point de vie, ainsi que 3 réapparitions.
Si le joueur tombe dans le vide ou que sa barre de vie tombe à 0, il perd une réapparition.
Lorsque le nombre de réapparition tombe à 0, le joueur retourne au menu principal et devra recommencer la map.

Déroulé d'une partie

Maps :

Lorsque la map commence, le compte à rebours se met en route, afin d'indiqué au joueur le temps restant qu'il doit rester en vie.

Pendant ce compte à rebours, différents mobs vont apparaître au fur et à mesure, avec un système de vague (non indiqué).
Lorsque le joueur détruit tous les mobs avant le prochain spawn, leur spawn va être avancer pour ne pas laisser de répis au joueur, et produire une tension constante.
Si le joueur n'a pas encore détruit tous les mobs, la nouvelle vague arrive, et le joueur se retrouve donc face aux ennemis de l'ancienne vague + ceux de la nouvelle.

Chaque vagues d'ennemis à une difficulté croissante et est de plus en plus difficile pour le joueur, que ce soit en nombre ou de part les mécaniques de gameplay des mobs.

Fin :

Lorsque le compte à rebours est terminé, le joueur doit détruire les ennemis restants. Une fois l'espace de jeu désaturé complètement, le joueur remporte la partie, et débloquent l'accès a une nouvelle map.

3C

Character

L'avatar de RoofRush est un humanoïde qui peut se déplacer de deux façons différentes, afin de remplir les conditions de rechargements de ses armes.

Armes :

L'avatar dispose de 3 armes, chacune disposants d'une condition de rechargement propre à elle, et d'un buff :

- Riffle : Fusil d'assault comportant 40 tirs par chargeur, cette arme est efficace à moyenne portée, et se recharge en effectuant un déplacement simple. Pour buff cette arme il faut etre au dessus d'une certaine vitesse et va augmenter la cadence de l'arme.

- Shotgun : Arme à courte portée contenant, 5 tirs par chargeur et effectue un knockback à chaque tir, cette arme se recharge en étant supérieur à la vitesse de déplacement simple. Le buff dépend de la vitesse maximum atteint par le joueur pendant le rechargement de celle-ci et va augmenter le nombre de balles tiré à chaque tir.

- Rocket : Arme à courte/moyenne portée, cette arme dispose de 3 tirs par chargeur et peut être utilisé par le joueur pour se propulser à l'aide de l'impacte, et se recharge en effectuant du bunny hop. Le buff va augmenter la zone d'impacte de chaque tir.

Déplacements :

L'avatar dispose de 2 façons de ce mouvoir dans l'espace de jeu :

- *le déplacement simple : avec ZQSD l'avatar se déplace à une valeur maximum de 800.*

- *Le bunny hop : en effectuant du bunny hop (enchainement de saut à la frame perfect au contact du sol) l'avatar va augmenter sa vitesse pouvant atteindre une valeur maximum de 1800.*

Réapparitions :

L'avatar dispose d'une jauge de vie, lorsque le joueur est bas en points de vie, l'UI «méta» va apparaître pour indiqué au joueur de se mettre à l'abris. Le joueur dispose de 3 réapparitions, tomber dans le vide, ou si la jauge de points de vie tombe à 0, le joueur perd une réapparition. Si le joueur à 0 réapparition, c'est un GAME OVER

Camera

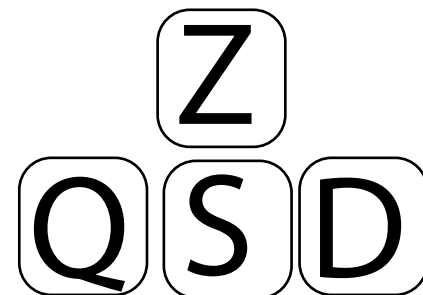
La caméra de RoofRush est en vue First Person Shooter.

Pour que le joueur puisse avoir une vision périphérique de son environnement pendant l'utilisation du bunny hop, nous avons mis le FOV de la caméra à 90.

Controller

Le mapping de RoofRush est simple. Le joueur se déplace avec ZQSD, change d'arme avec la molette de la souris. Deux input sont attribués pour le saut, afin de simplifier les 2 façons de bunny hop.

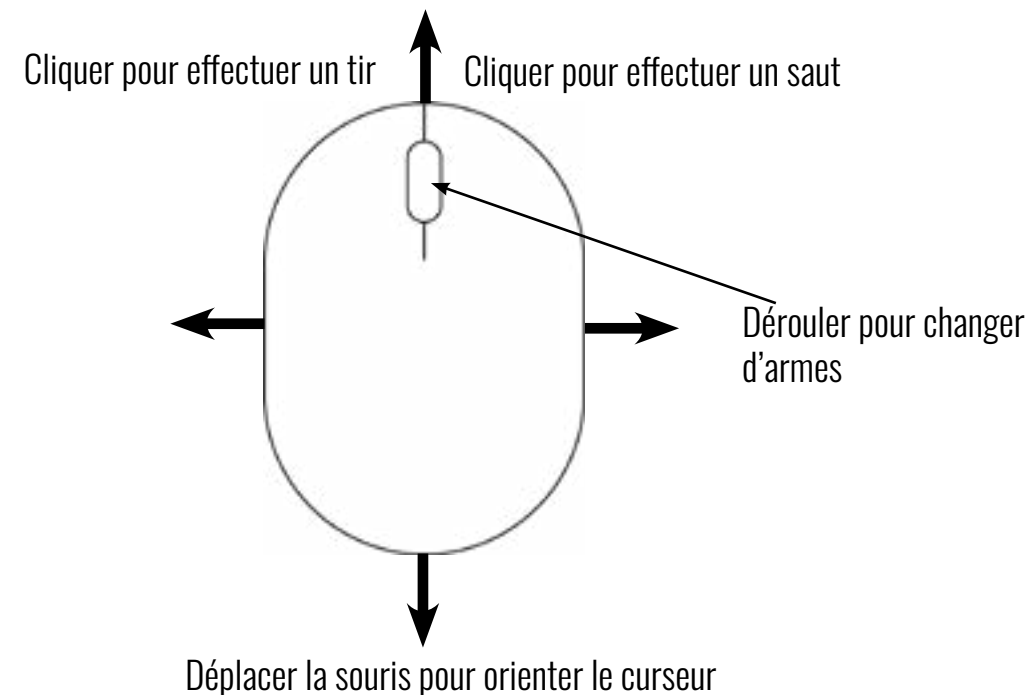
- Barre
- Clique droite (souris)



Appuyer pour déplacer l'avatar dans l'espace de jeu

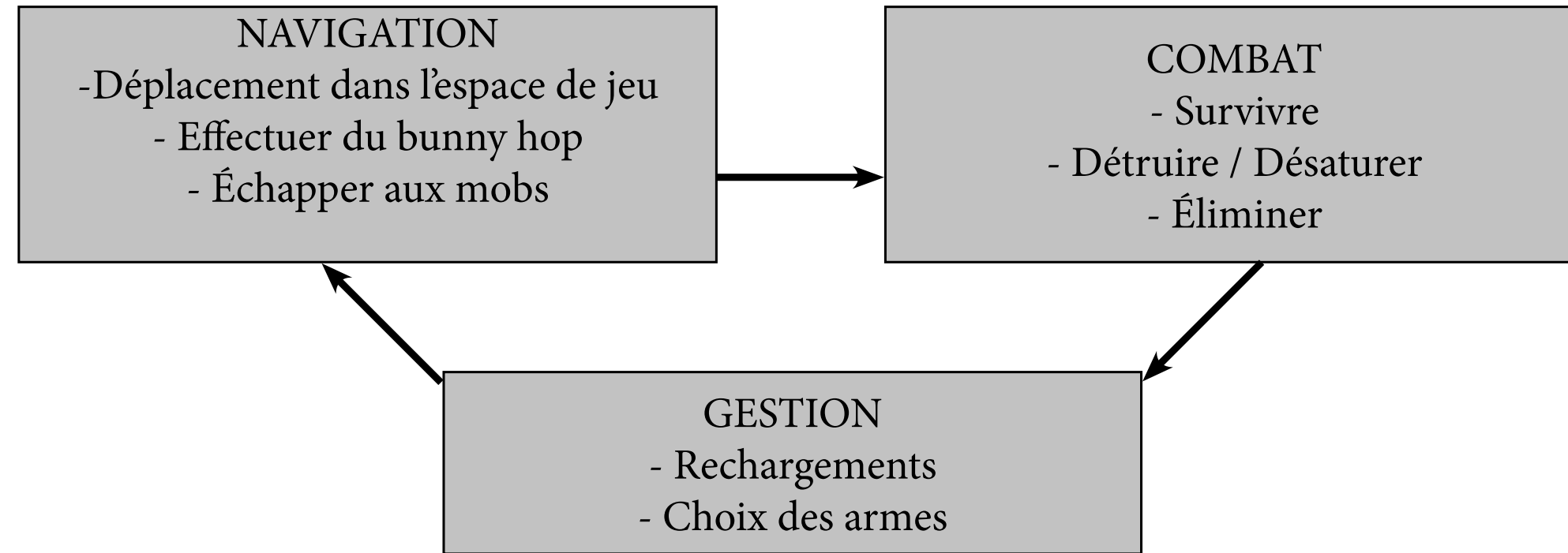
Barre espace

Appuyer pour effectuer un saut



UI affichant les armes, les conditions de rechargement, et si la condition est remplie.

Meta-boucle



Objectifs de jeu

Objectif de jeu Court :

Observer et analyser les patterns de l'environnement de jeu

Objectif de jeu Moyen :

Optimiser ses actions afin d'infliger le plus de désaturer l'espace de jeu avant le prochain respawn

Objectif de jeu Long :

Optimiser ses actions afin tenir 10 minutes en vie pour finir la map



Mécaniques - Déplacement

Déplacement

En maintenant les contrôles de déplacements (ZQSD + souris), l'avatar du joueur se déplace dans les 8 directions.



Mécaniques - Bunny Hop

Bunny Hop

Le bunny hopping est une technique de déplacement spécial d'un personnage utilisé dans un jeu de tir à la première personne (FPS).

Pour atteindre la vitesse maximum, les sauts doivent s'enchaîner et être coordonnés. Il est nécessaire de sauter au moment même où le personnage retombe au sol.



Mécaniques - Rechargement

Dans RoofRush, les armes nécessitent de remplir une condition pour être recharger. En effet, lorsque le joueur ne dispose plus de munition dans l'une de ses arme, il va devoir changer d'arme, et de remplir la condition de rechargement avec une autre arme.

Le joueur ne peut pas remplir la condition de rechargement de l'arme si il l'a tient en main.

les différentes conditions de rechargement :

- Riffle : Utilisé le déplacement normal (être à une vitesse inférieur à 800). Quand cette condition est remplie, le chargeur sera remplis de 10 balles chaque secondes durant la complétion de la condition.
- Shotgun : Être assez rapide (être à une vitesse supérieur à 1000). Quand cette condition est remplie, le chargeur sera remplis de 1 balles chaque secondes durant la complétion de la condition.
- Rocket : Faire du bunny hop (enchainer des sauts tout en étant supérieur à une vitesse de 1000). Quand cette condition est remplie, le chargeur sera remplis de 1 balles tous les 3 enchaînement de sauts.

L'UI nous indique quelles sont les conditions à effectuer pour recharger nos armes, et si la condition est en train d'être effectuer ou non :



Mécaniques - Riffle

Riffle : Fusil d'assault comportant 40 tirs par chargeur, cette arme est efficace à moyenne portée, et se recharge en effectuant un déplacement simple. Pour buff cette arme il faut etre au dessus d'une certaine vitesse et aura pour conséquence d'augmenter la cadence de l'arme.

Le riffle est considéré comme l'arme de base dans RoofRush, car elle est polyvalente. Elle est la seule arme présente dans le jeu qui n'impacte pas le déplacement du joueur.

la condition de buff est que le joueur doit effectuer un déplacement simple, et aura pour conséquence d'augmenter la cadence de tir de l'arme. Si le joueur est à l'arrêt, la cadence de tir revient à sa valeur d'origine.

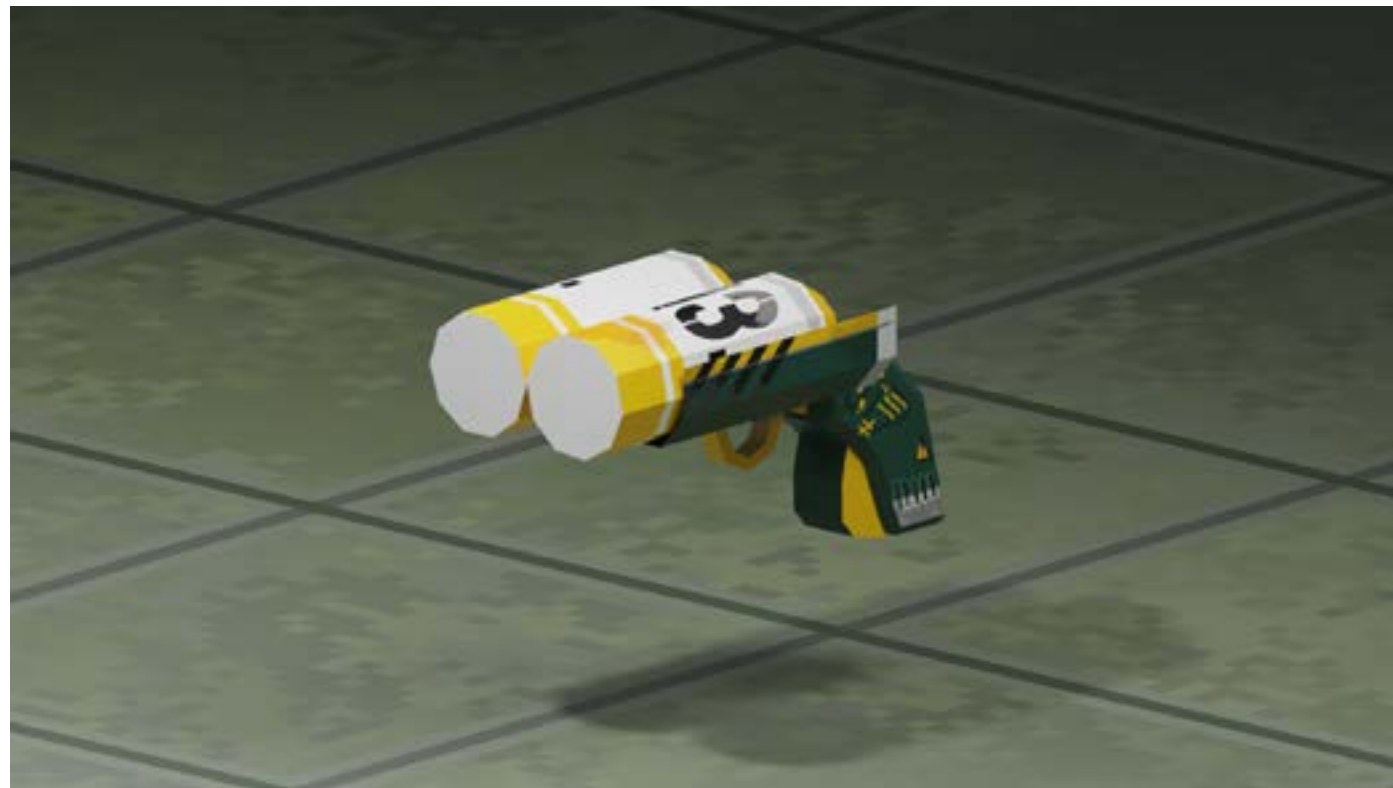


Mécaniques - Shotgun

Shotgun : Arme à courte portée contenant, 5 tirs par chargeur et effectue un knockback à chaque tir, cette arme se recharge en étant supérieur à la vitesse de déplacement simple. Le buff dépend de la vitesse maximum atteint par le joueur pendant le rechargement de celle-ci et va augmenter le nombre de balles tiré à chaque tir.

Le Shotgun est une arme redoutable a très courte portée, car elle inflige de très gros dégâts. Lorsque le joueur va tirer avec cette arme, il va subir un léger knockback, pouvant être utilisé comme un double-saut.

la condition de buff est que le joueur doit avoir une vitesse supérieur à la valeur 1000, et aura pour conséquence d'augmenter le nombre de balles tiré à chaque tir sur le prochain chargeur. Si le joueur change d'arme et revient sur le shotgun, le buff ne fonctionne plus et le nombre de balle tiré revient à ça valeur d'origine.



Mécaniques - Rocket

Rocket : Arme à courte/moyenne portée, cette arme dispose de 3 tirs par chargeur et peut être utilisé par le joueur pour se propulser à l'aide de l'impacte, et se recharge en effectuant du bunny hop. Le buff va augmenter la zone d'impacte de chaque tir.

Le Rocket est une arme de destruction qui inflige de lourd dégâts. Lorsque le joueur va tirer avec cette arme, il va subir un gros knockback, pouvant être utilisé pour atteindre des hauteurs habituellement non atteignable.

la condition de buff est que le joueur doit effectuer plusieurs saut d'affiler tout en étant à une vitesse supérieure à la valeur 1000, et aura pour conséquence d'augmenter la zone d'impacte à chaque tir sur le prochain chargeur. Si le joueur change d'arme et revient sur le rocket, le buff ne fonctionne plus et la zone d'impacte revient à ça valeur d'origine.



Bestiaires - Scrub

L'unité scrub est un ennemi qui apparaît en nombre, il se déplace en permanence vers la position du joueur, peut sauter de différente hauteur comme le joueur.

Lorsque cette ennemi est assez proche du joueur, il saute sur l'avatar et lui inflige des dégats.

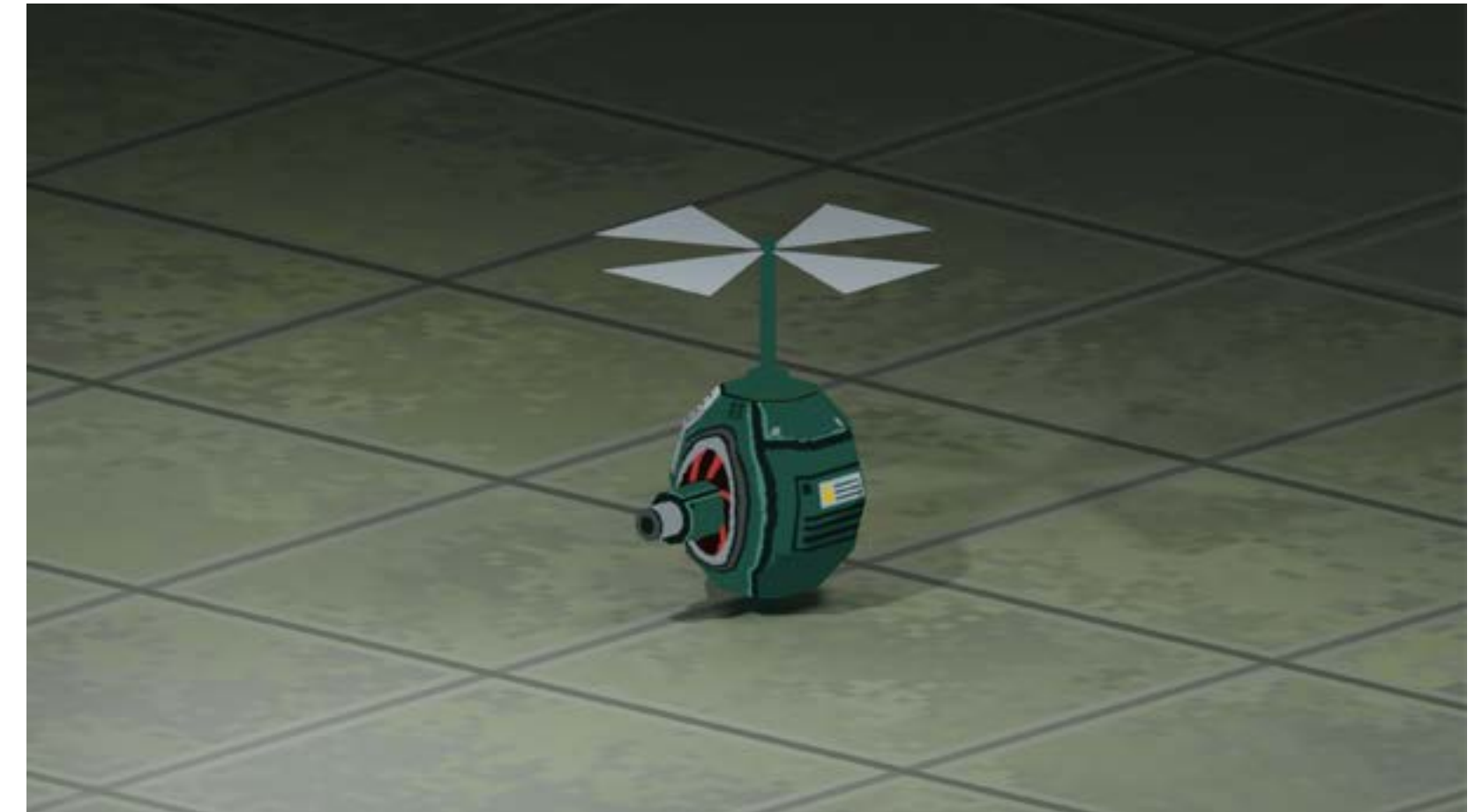
Cette ennemi est très présent dans le jeu.



Bestiaires - Distant

Le distant est un ennemi volant qui va prendre de la hauteur et de la distance entre l'avatar et lui-même pour lui envoyer des projectiles et lui infliger des dégats.

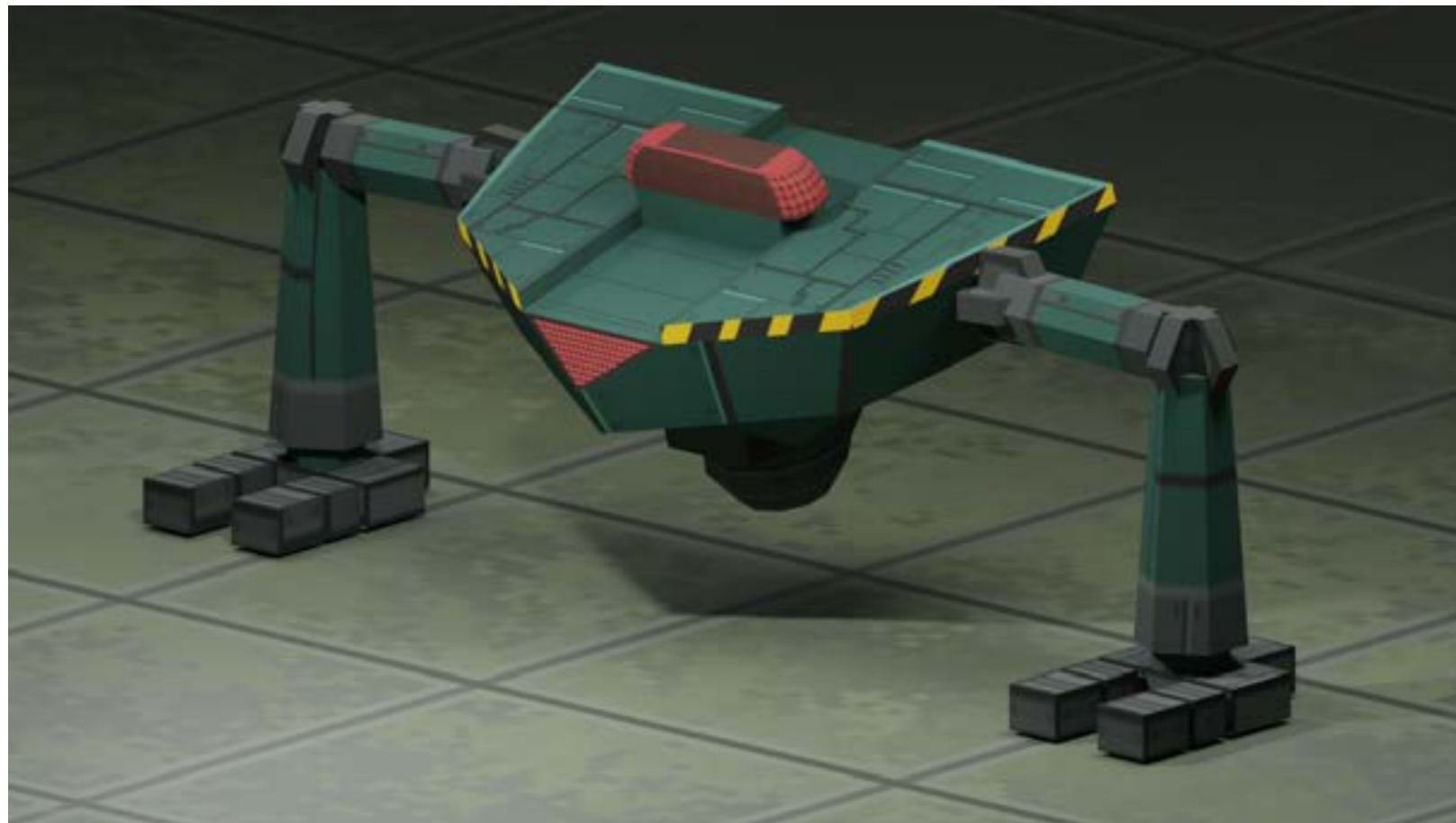
Cette unité est moyennement présente dans le jeu



Bestiaires - Goliath

Le goliath est un ennemi terrestre qui va couvrir une zone de l'espace de jeu, si le joueur s'approche trop du Goliath, il va poursuivre le joueur, et effectuer des charges pour tenter de faire tomber le joueur.

Cette unité est rare dans le jeu



Level Design - Explication du niveau

RoofRush est un FPS Fast Arena Shooter. Le joueur devra affronter des vagues d'ennemis qui apparaîtront à plusieurs positions sur la map, le joueur a donc pour objectif de tous les éliminer sans se faire tuer. Pour cela, le joueur possède plusieurs déplacements : le déplacement normal, le saut, de l'air contrôles et ainsi qu'un déplacement technique le bunny-hop qui va permettre au joueur de gagner en vitesse.

Les vagues sont composées de plusieurs types de monstre : Un volant qui attaque à distance et qui esquive de temps en temps les projectiles du joueur, un imposant qui cherchera toujours à charger le joueur lorsque celui-ci est proche et dans son champ de vision, et finalement un de type "trash" qui pourchasse le joueur et fait des bonds vers celui-ci pour essayer de l'attaquer.

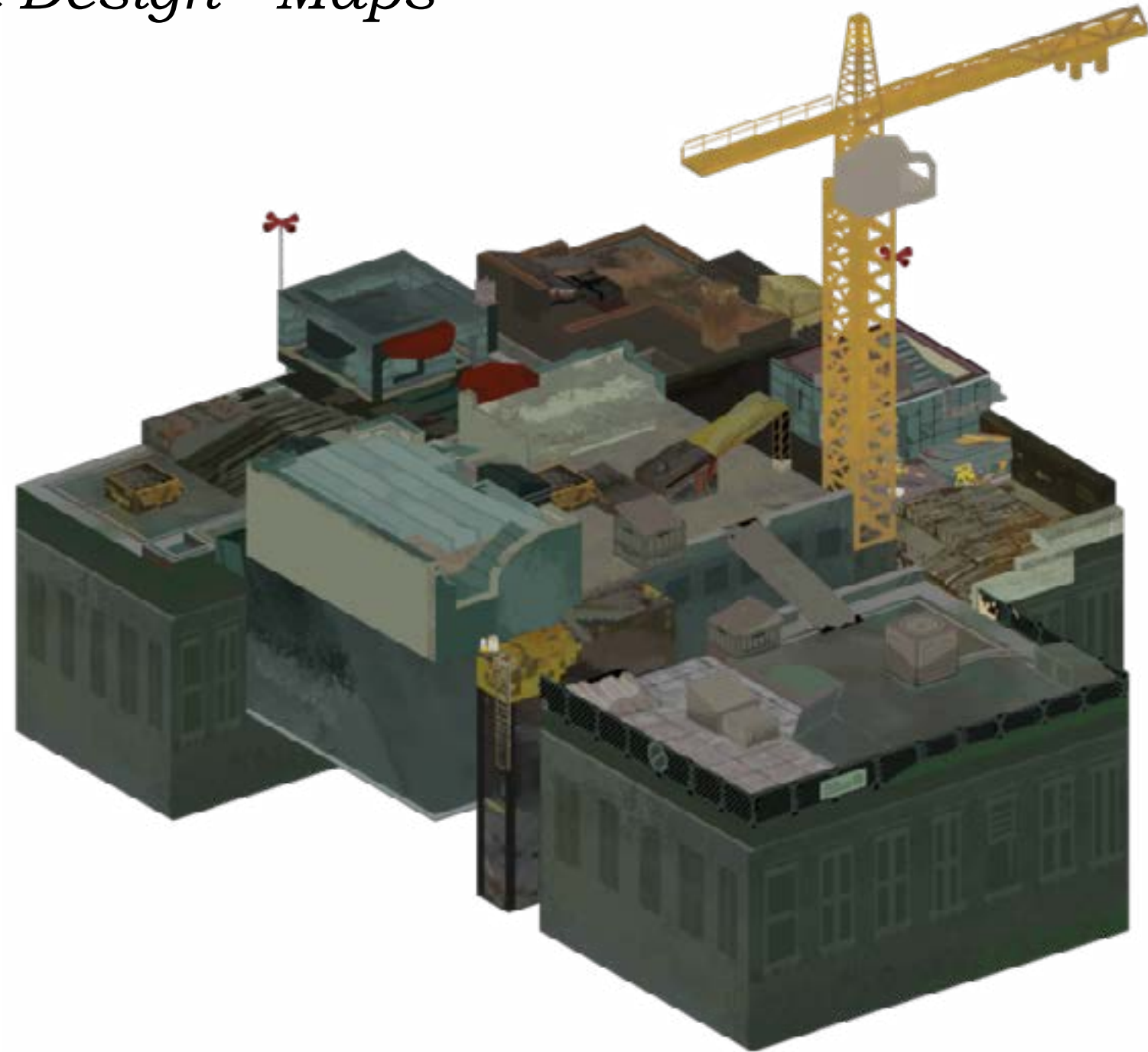
Le niveau est composé de plusieurs niveaux d'élévation et de surfaces de taille différentes, séparés par d'énormes gouffres. Ces différentes surfaces sont connectées par un "hub", ce qui permet au joueur de se repérer et de créer des "loops". Rebords, caisses, rampes, le joueur va pouvoir grâce au bunny-hop se déplacer librement à grande vitesse (S'il a le skill).

L'univers se passe dans la journée en haut des toits de grand gratte-ciels en plein centre ville d'une ville urbaine post-apocalyptique dégradé par de nombreux graffitis anti-politique

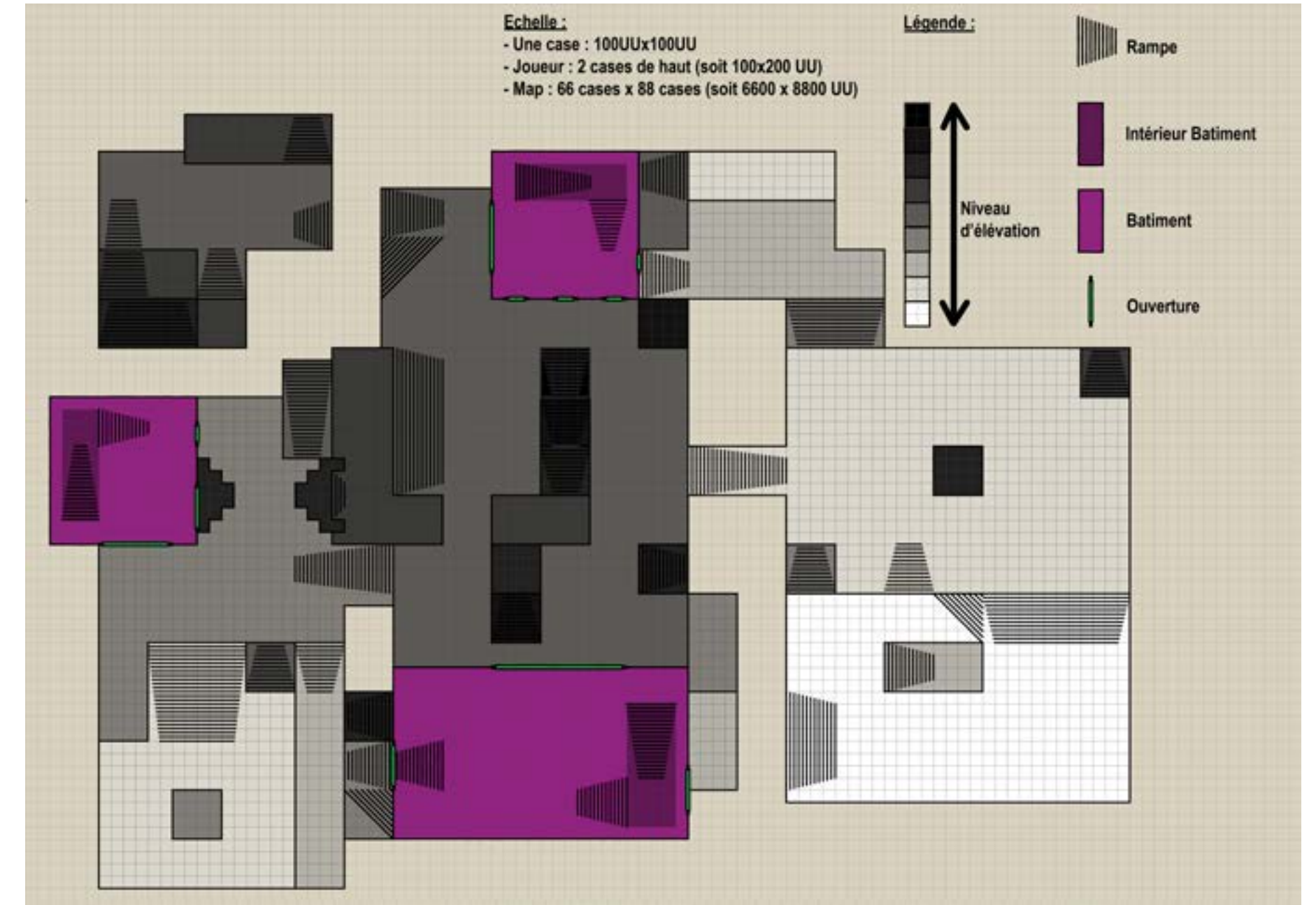
Intentions :

- Déplacement libre, rapide et nerveux
- Level Design qui engage a bunny-hop
- Les armes qui permet de dériver le déplacement dans le LD

Level Design - Maps

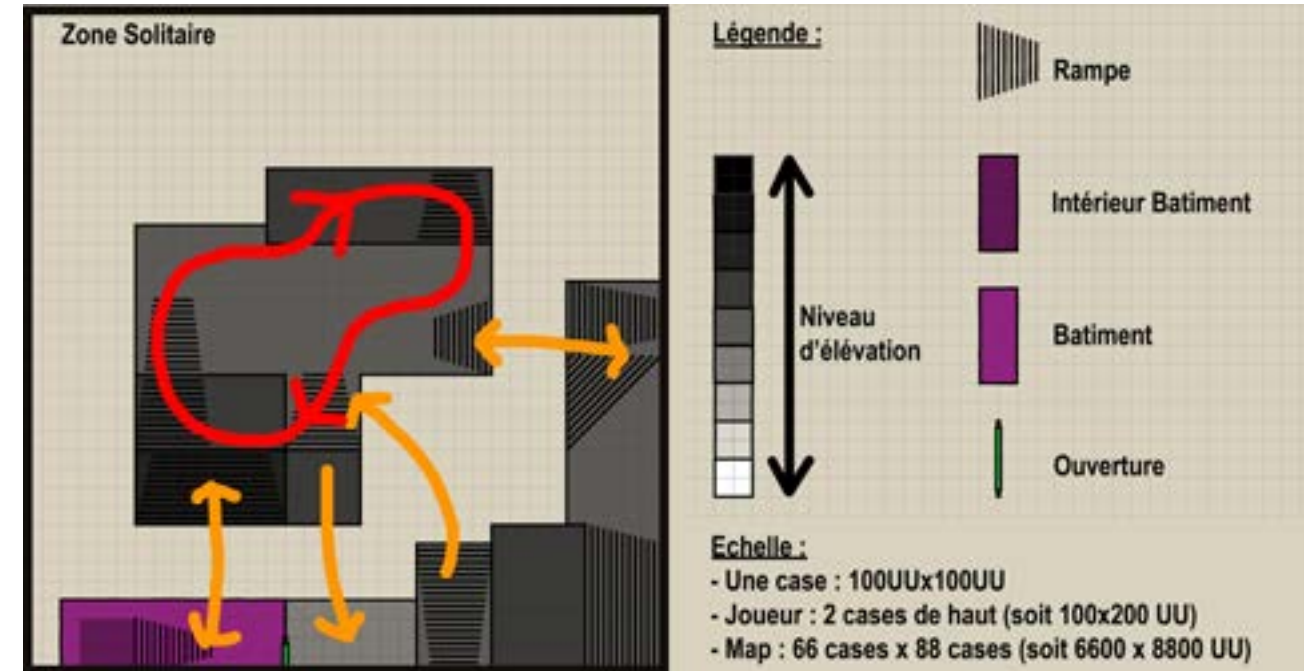


Level Design - Maps



Level Design - Les zones

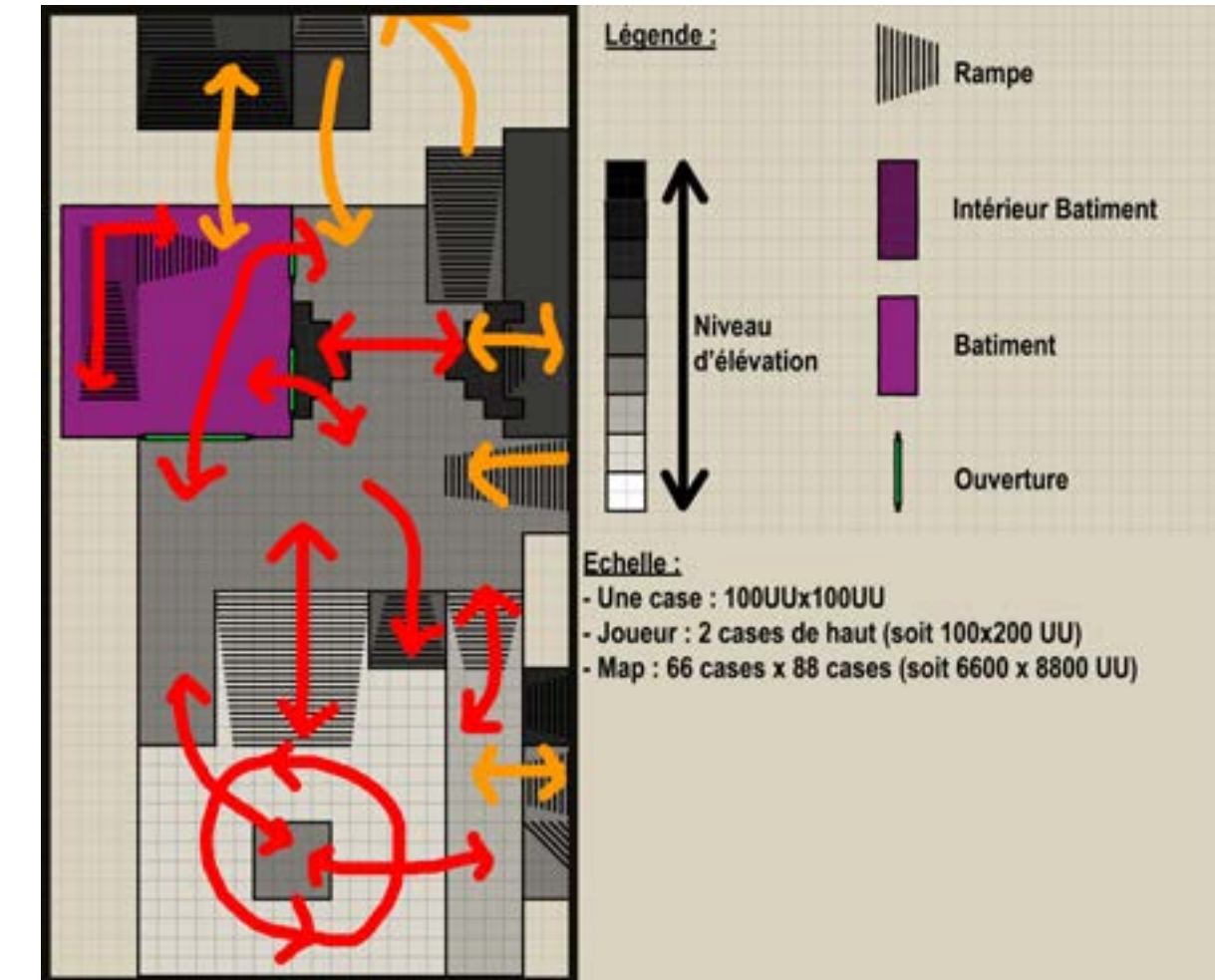
La map de RoofRush est découpée en 4 zones : Zone Solitaire, Zone du milieu, Zone de droite et Zone de gauche. Chacune des zones a une importance capitale, et permet une homogénéité de la map, ainsi que de bonne circulation. Chacune des zones présente des modules propres à la zone, permettant au joueur d'effectuer des déplacements uniques à chaque zone.



Zone solitaire, disposant de 3 élévations. Cette zone se caractérise par sa taille, elle est transitoire entre la zone du milieu et la zone de gauche. Elle n'est pas directement reliée aux autres zones, mais permet aux joueurs de la traverser de différentes façons.

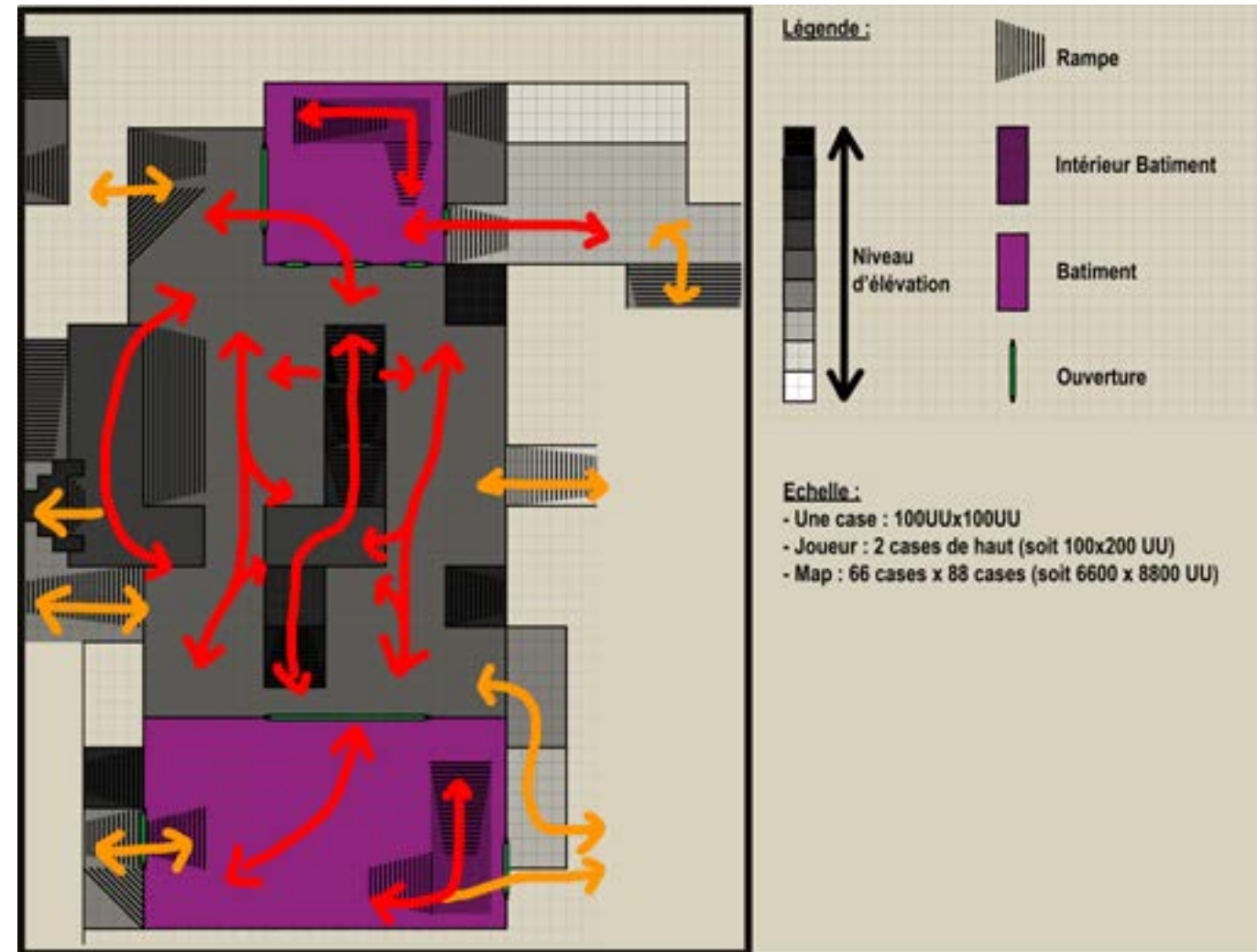
Level Design - Les zones

Zone gauche, disposant de 5 élévations différentes. Elle se caractérise par ses différentes lignes de vue que peut prendre le joueur, que ça soit au sol, mais aussi en hauteur. Elle est reliée à la zone solitaire et à la zone du milieu. Ses circulateurs sont nombreux et permettent aux joueurs de se positionner stratégiquement et rapidement avec les différents modules présents dans cette zone.



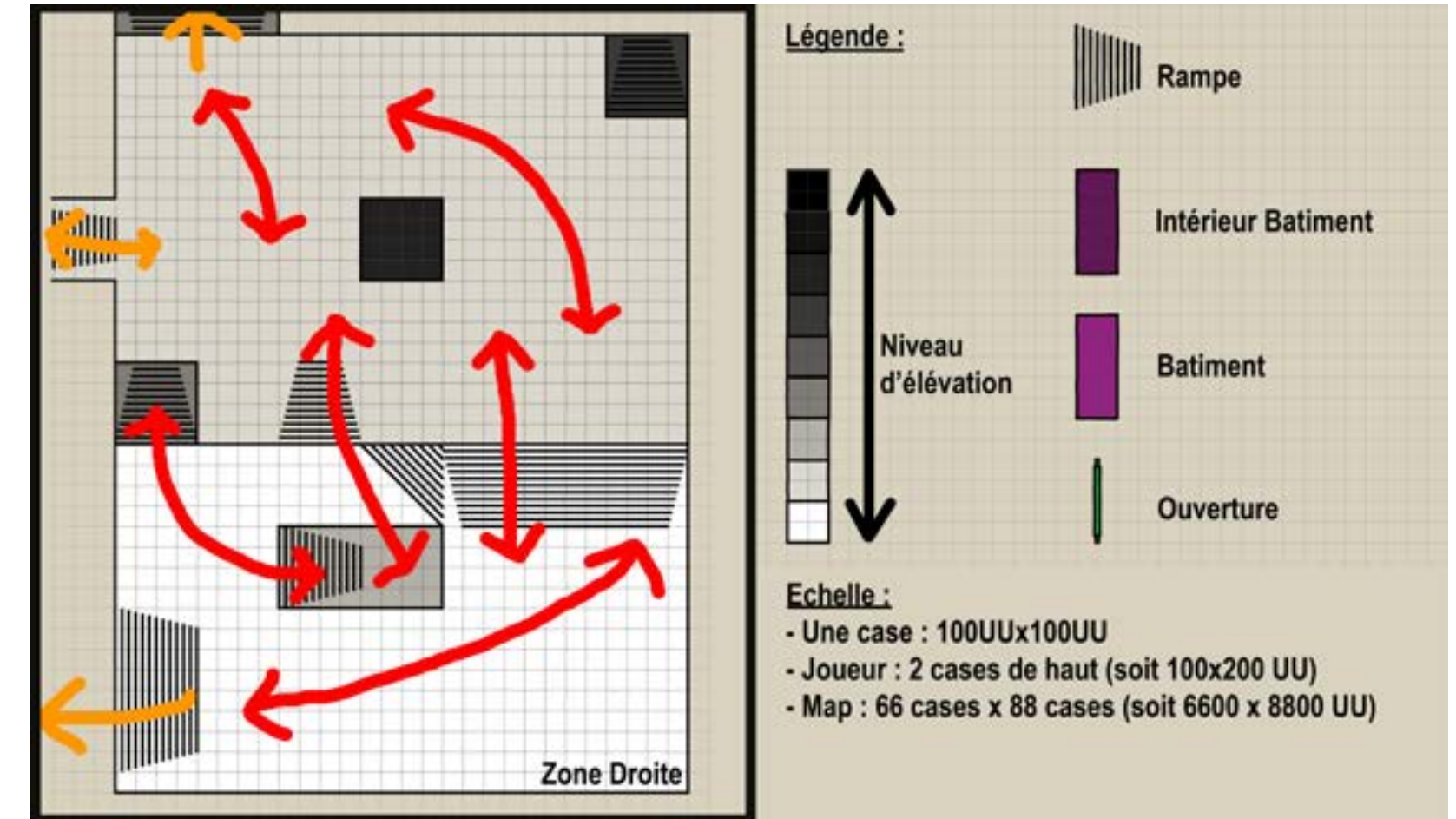
Level Design - Les zones

Zone du milieu, disposant de 8 élévations. Elle est la zone principale de la map, et se caractérise par ses nombreux sens de circulations qui permettent de créer une connexion entre chaque zone de la map. Ses nombreuses élévations permettent au joueur de prendre du recul et de la hauteur, afin de planifier ses futurs déplacements.



Level Design - Les zones

Zone de droite, disposant de 5 élévations. Elle est plus plate que les autres, et ajoute de l'espace au joueur. Elle n'est reliée qu'à la zone du milieu.



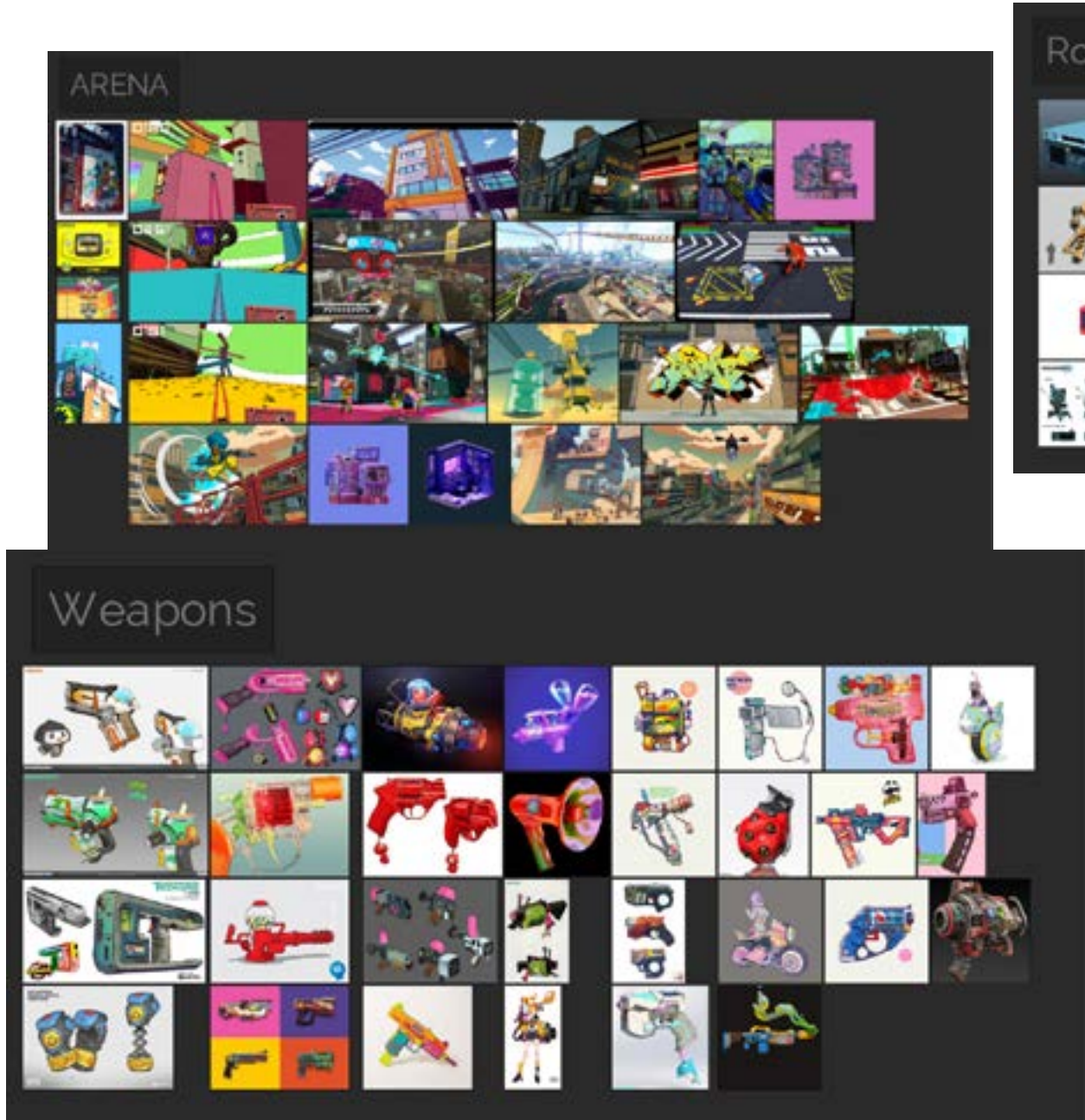
UI - Armes



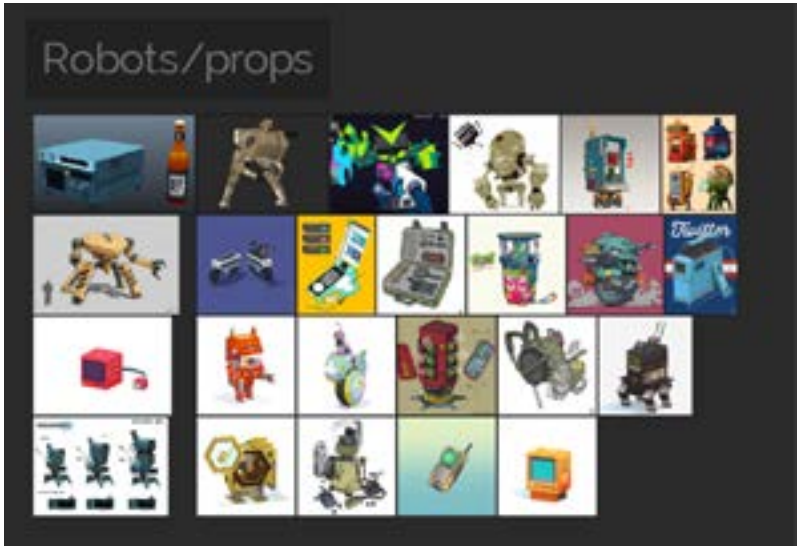
UI - Points de vie



Références de Design Visuel



Intentions de Design Visuel



Itérations de Design Visuel

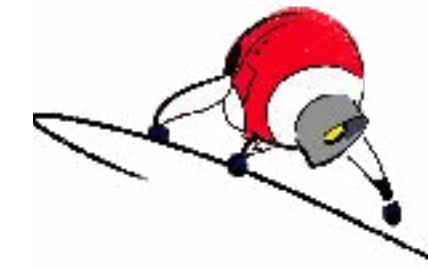
Itérations de Design Visuel



Le robot Distance est pyramidal, il a un petit canon à l'avant pour ses attaques et se déplace avec 4 jambes mécaniques. La particularité de ce robot est sa capacité à faire reculer une unité ennemie ou alliée. Pour illustrer cette action le robot projette un faisceau de lumière "laser", il sera bleu pour les alliés et rouge pour les ennemis.

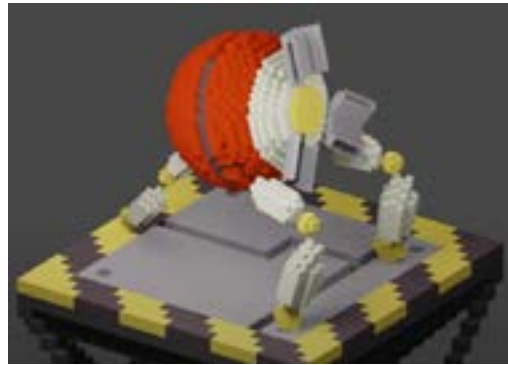


Le robot Soutien est solide, cubique. Il possède des compartiments déployables selon l'action demandée (jambes pour le déplacement, bras pour le grab etc.).

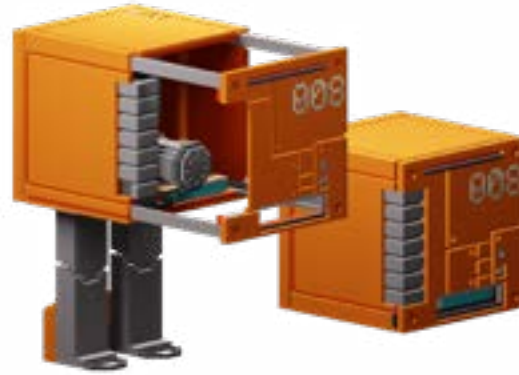


Le robot Corps-à-corps est sphérique et à 3 appendices à roues pour des déplacements rapides. Il utilise ces 3 jambes mécaniques pour les attaques simples et son tube avant comme hélice.

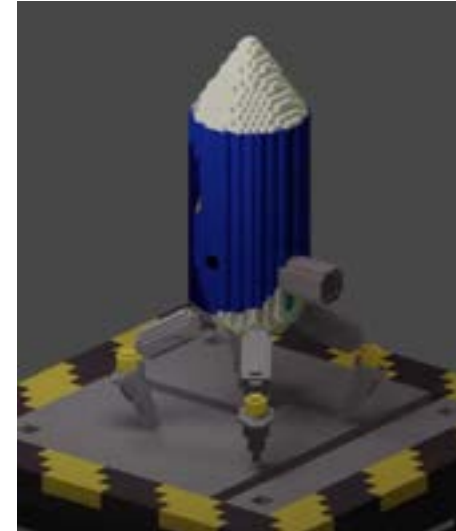
Design Visuel Final



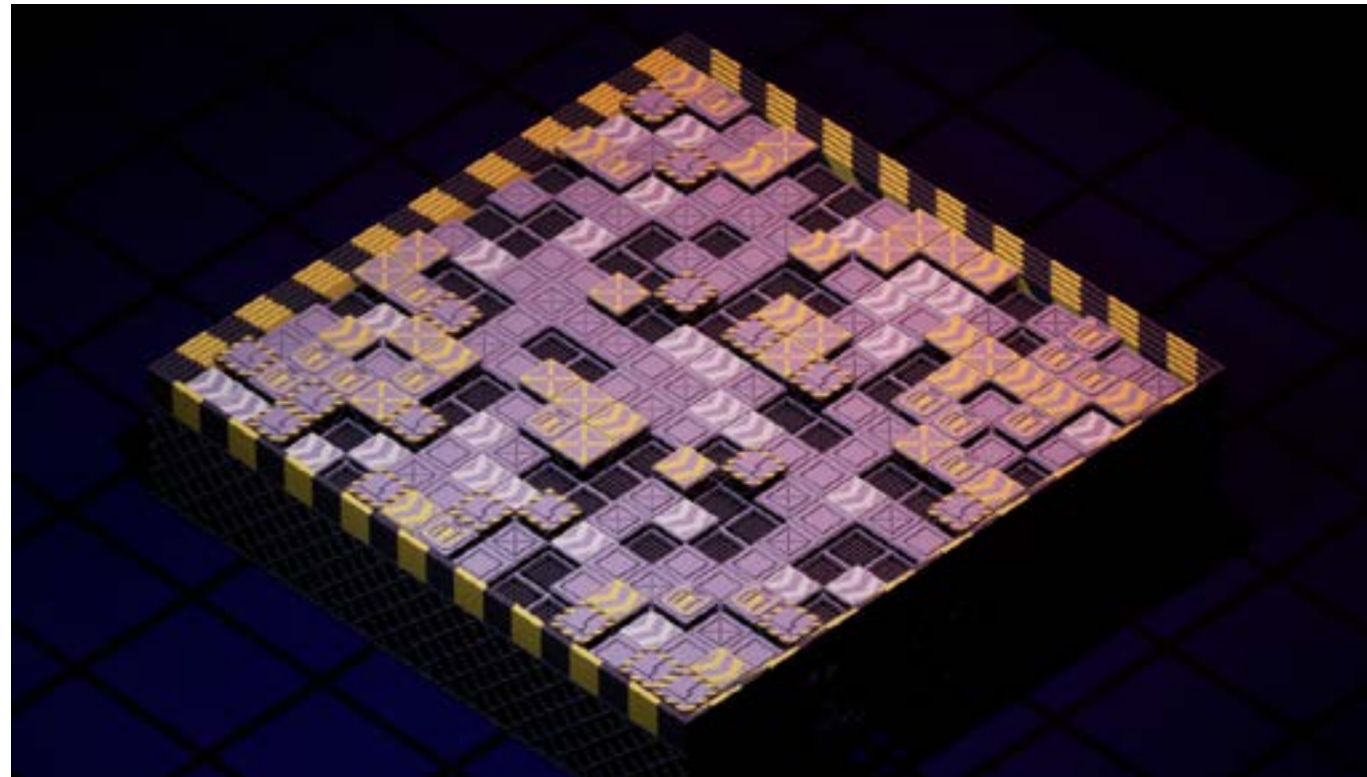
Robot Corps-à-corps



Robot Soutien



Robot Distance



Arène

Intentions de Design Sonore

Ayant une arène composée de dalles ainsi que de robots, il était important pour nous de faire ressortir le côté métallique de l'environnement.

De plus, nous voulions que les sons liés aux robots permettent non seulement de comprendre leurs différents outils, mais également de renforcer les parties qui les caractérisent afin d'accentuer leur design asymétrique.

Dans la globalité, nous avons pensé à réaliser des sons courts, la principale raison est que nous voulions une bonne compréhension des actions des 2 joueurs et ainsi éviter une superposition de trop de sons si les joueurs venaient à jouer rapidement leurs tours.

Côté UI, nous nous sommes fait la même réflexion, en essayant d'avoir des sons de la durée d'un clic, tout en associant à certains d'entre eux des variations permettant de venir compléter un univers métallique et robotique.

Enfin pour l'Annonceur, nous voulions faire en sorte que celui-ci paraisse robotique pour encore une fois venir coller à l'univers de notre jeu. Ce dernier interviendra lors des différentes actions des 2 joueurs.

EventList Sonore - Tableau SFX

Après avoir établi le core design du projet et son esthétisme, nous avons mis en place un tableau SFX permettant de catégoriser les différents effets sonores pour notre projet :

SFX						
Remarque	Nom Event	Description Composition	Parametre	Condition Trigger	Spatialisation	Loop ?
ARENA						
Environnement						
Validé & Intégré	Env_General	Vent	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	AC_Fan_1	Bruit de ventilation extérieur de bâtiment	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	AC_Fan_2	Bruit de ventilation extérieur de bâtiment	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	HVAC_Motor_1	Bruit de cube électrique extérieur de bâtiment	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	HVAC_Motor_2	Bruit de cube électrique extérieur de bâtiment	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	Birds_1	Bruit d'oiseau ville	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	Birds_2	Bruit d'oiseau parc	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	Birds_3	Bruit d'oiseau parc lointain	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	Traffic_1	Bruit de trafic parc lointain	/	Se joue des lors que le jeu se lance	2D	☒
Validé & Intégré	Traffic_2	Bruit de trafic + foule lointaine	/	Se joue des lors que le jeu se lance	2D	☒
Impact/Bullet						
Validé & Intégré	Impact_Bullet	Sons aigu électrique impactant	Parametre qui modifie le son selon la texture de l'obstacle	Se joue lorsque la balle touche un obstacle	3D	☐
Validé & Intégré	Impact_Rocket	Son d'explosion, son d'explosion électrique, son de glitch électrique, son électrique	/	Se joue lorsque la rocket touche un obstacle	3D	☐
Character						
Hurt						
Pas fait	Dmg_Hit		/	Se joue lorsque l'avatar perd des points de vie	2D	☐
Move						
Validé & Pas encore Intégré	FootStep	Bruit de pas sur surface solide en pierre / bitume	/	Se joue à une frame de l'animation de déplacement de l'avatar		☐
Validé & Intégré	Jump	Bruit d'impulsion grave, son électrique et glitch	Parametre qui modifie le son selon la vitesse. Plus le joueur va vite plus les sons vont augmenter en volume	Se joue lorsque le joueur saute	2D	☐
Validé & Intégré	Moving	Bruit de vent	Parametre qui modifie le son selon la vitesse. Plus le joueur va vite plus le son de vent va augmenter	Se joue des lors que le jeu se lance	2D	☒

EventList Sonore - Tableau SFX

Enemies						
TrashMob						
Validé & Intégré	Trash_Ask	Bruit métallique, impact d'objets métallique	/	Se joue lorsque le TrashMob réussi à toucher le joueur avec son attaque	3D	☐
Validé & Intégré	Trash_WooshAsk	Whoosh de vent d'un mouvement rapide	/	Se joue lorsque le TrashMob attaque le joueur	3D	☐
Validé & Intégré	Trash_Die	Explosion, impact d'objet métallique	/	Se joue lorsque le TrashMob meurt	3D	☐
Validé & Intégré	Trash_Hit	Bruit / voix robotique	/	Se joue lorsque le trashmob est touché par un projectile du joueur	3D	☐
Pas fait	Trash_Jump		/		3D	☐
Validé & Intégré	Trash_Move	Bruit d'une roue qui roule sur du bitume, son d'électrique, son 'humming-buzz'	/	Se joue des lors que le TrashMob spawn	3D	☒
FlyMob						
Validé & Intégré	Fly_Die	Explosion, impact d'objet métallique	/	Se joue lorsque le FlyMob meurt	3D	☐
Validé & Intégré	Fly_Hit	Bruit / voix robotique	/	Se joue lorsque le trashmob est touché par un projectile du joueur	3D	☐
Validé & Intégré	Fly_Move	Bruit d'hélices	/	Se joue des lors que le FlyMob spawn	3D	☒
Validé & Intégré	Fly_Shoot	Bruit de feu d'une arme d'assaut	/	Se joue lorsque le FlyMob tire	3D	☐
TankMob						
Pas fait	Tank_Ask			Se joue lorsque le TankMob attaque le joueur	3D	☐
Pas fait	Tank_Dash			Se joue lorsque le Tank dash vers le joueur	3D	☐
Pas fait	Tank_Die			Se joue lorsque le TankMob meurt	3D	☐
Validé & Intégré	Tank_Hit	Bruit / voix robotique	/	Se joue lorsque le TankMob perd des points de vie	3D	☐
Validé & Intégré	Tank_Move	Bruit de système hydraulique, impact de bruit de métallique, petite explosion et soufflement de poussières	/	Se joue à une frame d'animation de déplacement du TankMob	3D	☐

EventList Sonore - Tableau SFX

Weapon						
Rifle						
Validé & Intégré	Rifle_Fire	Son de tir d'une arme d'assaut + son d'électricité et vibration électrique	REFERENCE	Se joue lorsque "Rifle_Shoot" est lancé		☐
Validé & Intégré	Rifle_Start	Son du trigger d'une arme d'assaut	REFERENCE	Se joue lorsque "Rifle_Shoot" est lancé		☐
Validé & Intégré	Rifle_Shoot	Prends en compte les deux références cité au dessus	Parametre qui va rendre muet l'event Rifle_Fire si le joueur n'a plus de munition	Se joue lorsque le joueur tire avec l'arme Rifle	20	☐
RocketLauncher						
Validé & Intégré	RocketLauncher_Fire		REFERENCE	Se joue lorsque "RocketLauncher_Shoot" est lancé		☐
Validé & Intégré	RocketLauncher_Start	Son du trigger d'un rocket launcher	REFERENCE	Se joue lorsque "RocketLauncher_Shoot" est lancé		☐
Validé & Intégré	RocketLauncher_Shoot		Parametre qui va rendre muet l'event RocketLauncher_Fire si le joueur n'a plus de munition	Se joue lorsque le joueur tire avec l'arme RocketLauncher	20	☐
Projectile_Rocket						
Validé & Intégré	RocketLoop	Bruit de fusée, son électrique, bruit de glitch		Se joue lorsque le projectile est spawné dans l'espace de jeu	20	☒
PumpActionLauncher						
Validé & Intégré	RocketPump	Bruit d'engrenage, bruit de soufflement, bruit électrique		Se joue a une frame d'animation de recharge du RocketLauncher	20	☐
ShotGun						
Validé & Intégré	Shotgun_Fire	Son de tir d'un shotgun + son petite explosion électrique	REFERENCE	Se joue lorsque "Shotgun_Shoot" est lancé		☐
Validé & Intégré	Shotgun_Start	Son du trigger d'un shotgun	REFERENCE	Se joue lorsque "Shotgun_Shoot" est lancé		☐
Validé & Intégré	Shotgun_Shoot	Prends en compte les deux références cité au dessus	Parametre qui va rendre muet l'event Shotgun_Fire si le joueur n'a plus de munition	Se joue lorsque le joueur tire avec l'arme Shotgun	20	☐
PumpActionShotgun						
Validé & Intégré	ShotgunFirstPump	Son d'un pumpaction d'un shotgun		Se joue a une frame d'animation de recharge du Shotgun	20	☐
Validé & Intégré	ShotgunSecondPump	Son d'un pumpaction d'un shotgun		Se joue a une frame d'animation de recharge du Shotgun	20	☐

Création sonore

Avant tout, le sound design de Roofrush a été entièrement réalisé par Charles Barrier avec plusieurs logiciels tel que : FMOD et Reaper. Toute l'intégration sonore sur le projet a également été réalisée par Charles Barrier avec l'aide du Game Programming du projet, Antoine Dias.

L'environnement :



Étant donné que l'univers se passe dans une ville urbaine surpeuplé, entouré de bâtiments délabrés, sur des toits de bâtiments. Les sons d'ambiance doivent être adaptés pour immerger le joueur dans cet univers. Nous avons donc des sons de trafics, d'oiseaux, de foules, de vents, des sons plus précis situés sur les toits comme des ventilations, des blocs centrale électrique, etc.

Création sonore

L'avatar :

Le joueur incarne un jeune voyou humain qui traîne sur les toits des immeubles, vandalisant la propagande et se rebellant contre les robots en utilisant une technique d'armement et de déplacement innovante.

Pour être immergé dans la peau de l'avatar, et aussi ressentir la vitesse. Les sons principalement de sauts et de vents sont liés à la vitesse. Plus le joueur va prendre de la vitesse, plus ces sons vont changer.

Les sons de sauts sont généralement créé avec des sons d'impulsion grave, son électrique et glitch qui vont être accentué avec la vitesse. Le sons de vent aussi va changer en volume selon la vitesse.



Création sonore

Les armes :

Le joueur a à sa disposition 3 armes différentes qui ont des spécificités bien différentes. Une arme d'assaut, un fusil à pompe et un lance-roquette. Celles-ci sont construites principalement de métal et d'électricité. Et tire des projectiles électriques.

Tous les sons de ces armes, pour les tirs, ont été créés principalement d'explosion, d'impact lourd, d'électricité, de glitch.

Pour les projectiles ainsi que l'impact du projectile sur les obstacles, pour le fusils d'assaut et le fusil a pompe, principalement d'électricités aigu. Et pour le lance-roquette d'un son de fusée, de glitch et d'électricité grave avec un ajout d'un son d'explosion électrifié lors de l'impact de celle-ci.

Il y a aussi des sons qui caractérise le rechargement, notamment pour le fuils à pompe et le lance-roquette, qui définit le fonctionnement de ces armes.



Création sonore

Les mobs :

Le joueur va faire face à plusieurs robots provenant de la police dans lequel le joueur est plongé. Nous avons, dans l'ordre, le Scrub, le Fly et finalement le Goliath.

Chaque mobs à ses propres spécificités et peuvent être identifiées par les différents sons qu'ils émettent.

En général pour le bestiaire j'ai voulu mettre des sons à la fois robotisés, réalistes mais aussi caractéristiques (Faire en sorte que dès que le joueur écoute le son, sans voir l'entité, il sait à quoi il à affaire).

De ce fait, les sons de déplacement caractérisent bien les différents mobs. Le trashmob à des sons de mouvement de roue, le Fly a des sons d'hélices et le Goliath des sons hydrauliques avec des sons de pas d'impact de métal lourd.

Tous ces mobs émettent également des bruits robotiques lorsque celles-ci prennent des dégâts.



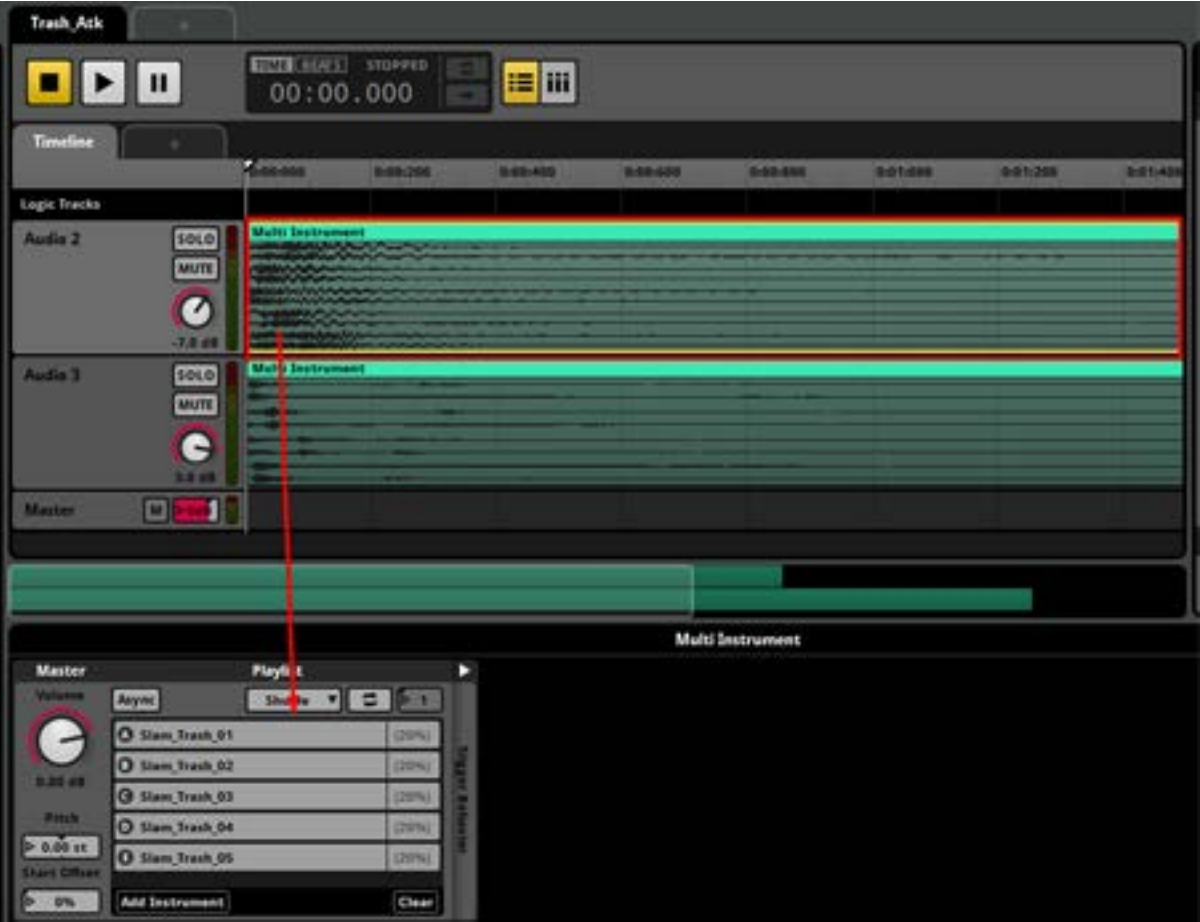
Création sonore

FMOD - Mécaniques Sonores :

Après avoir créé tous les sons, nous les avons importé sur FMOD en utilisant des mécaniques que FMOD nous propose :

- Multi-Instrument :

Nous utilisons des multi-instruments, ce sont plusieurs sons dans une liste. Lorsque le multi-instruments joue, un son de la liste est joué. Nos multi-instruments sont dans le même style et thème : des multi-instruments de tirs, pas, coups, mort, etc.



Création sonore

- Compressor et 3EQ :

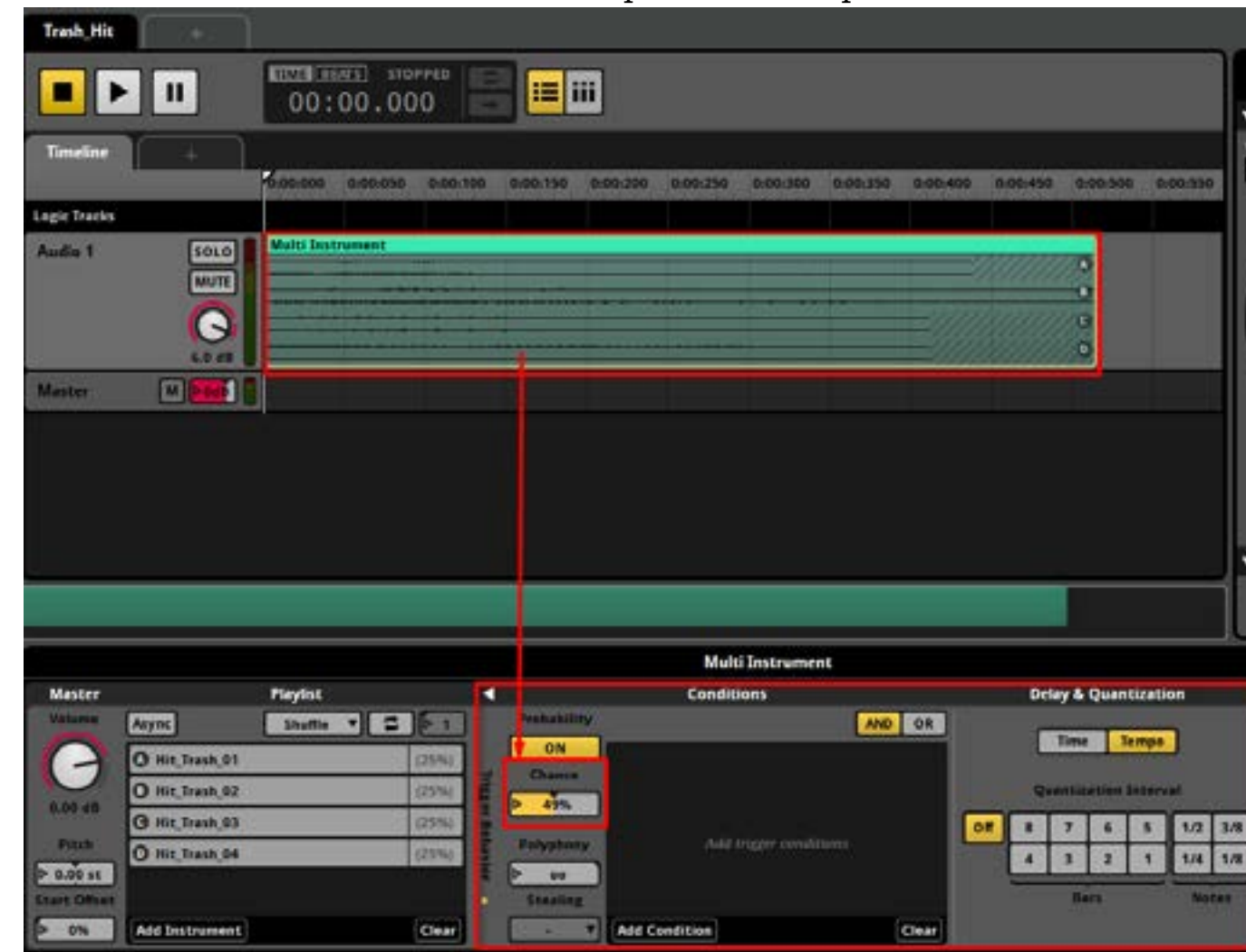
Nous avons utilisé aussi plusieurs fois les effets de compressor et 3-EQ afin de mieux accentuer l'impact des différents effets sonores explosif ce qui les rends plus en valeur



Création sonore

- Probabilités :

Pour tous les sons répétitifs, hormis les sons de tirs, nous avons mis en place une probabilité d'activation d'un multi-instruments donné, notamment les sons saut, cris de robot lorsque celui perd des points de vie. Cela permet de réduire le nombre de ces sons et éviter les répétitions nonplaisantes à l'oreille.



Création sonore

- Référence :

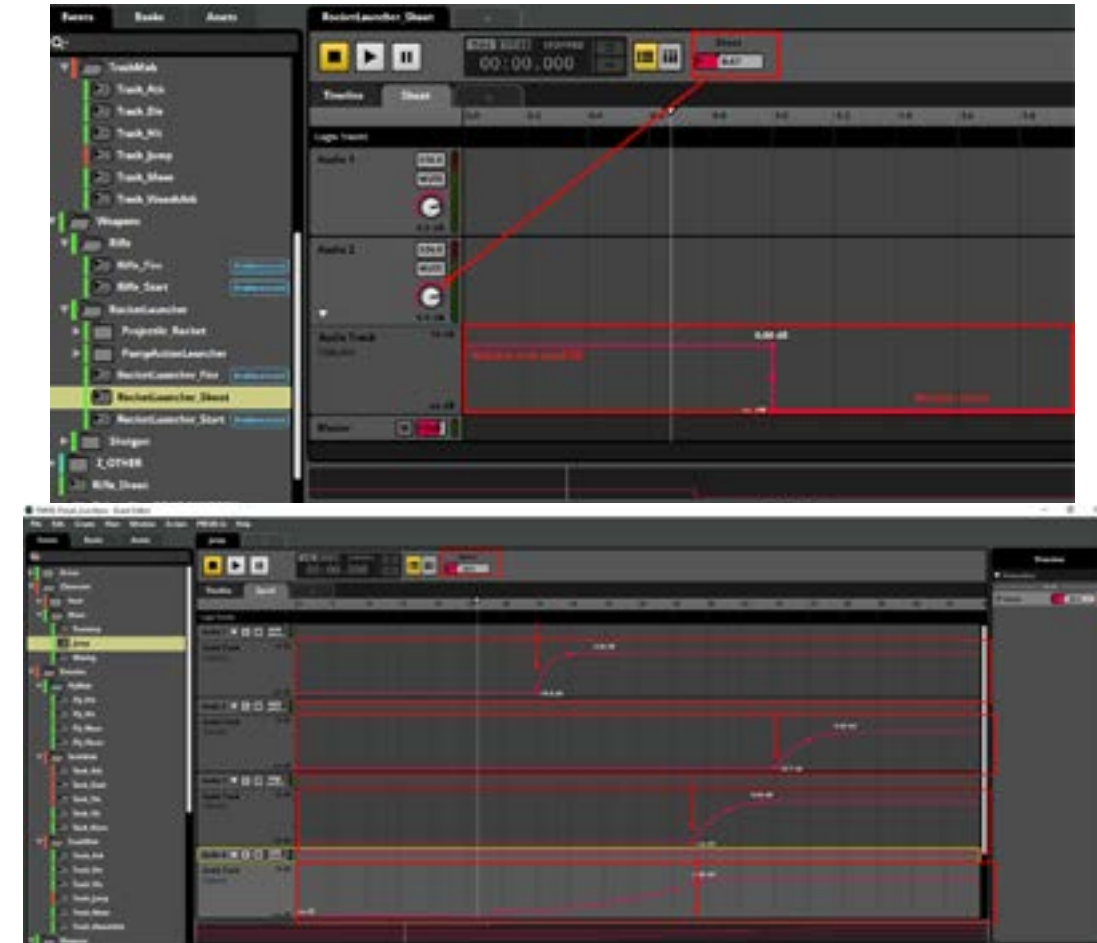
Nous utilisons les References sur FMOD, Cela permet à un event de créer et d'arrêter des instances des autres événements qu'il référence. De cette manière, il est possible de combiner un certain nombre d'événements différents pour obtenir une vaste gamme de résultats. Nous utilisons cette mécanique de FMOD principalement pour les tirs des armes.



Création sonore

- Parameters :

Nous utilisons les Parameters sur FMOD, le seul parameters important sur notre projet sons est le Parameter de Vitesse que beaucoup de nos events sons ont. Les parameters sont des variables qui peuvent être modifiées en temps réel dans le moteur de jeu, notre parameters de vitesse change en fonction de la vitesse en termes de pourcentage (Variable allant de 0 à 100, plus on va vers 100 plus on est rapide et inversement). Les events notamment de saut, de vents, de tirs changent considérablement lorsque le joueur prend de la vitesse.



Création sonore

- Doppler :
Ainsi pour rebondir sur la vitesse, beaucoup de nos events sons ont été mis en place avec un doppler. Ce qui facilite encore plus l’immersion dans notre projet.



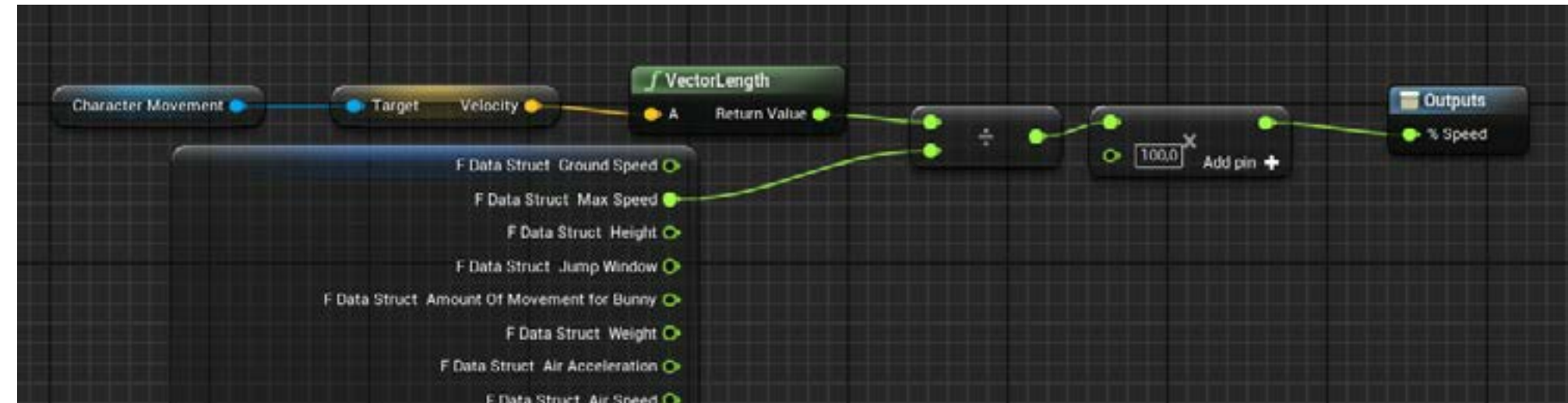
Création sonore

- UNREAL - INTÉGRATION EFFETS SONORES :
Pour l’intégration des sons dans l’environnement, nous avons tout simplement glisser déposer les events définits pour l’univers :



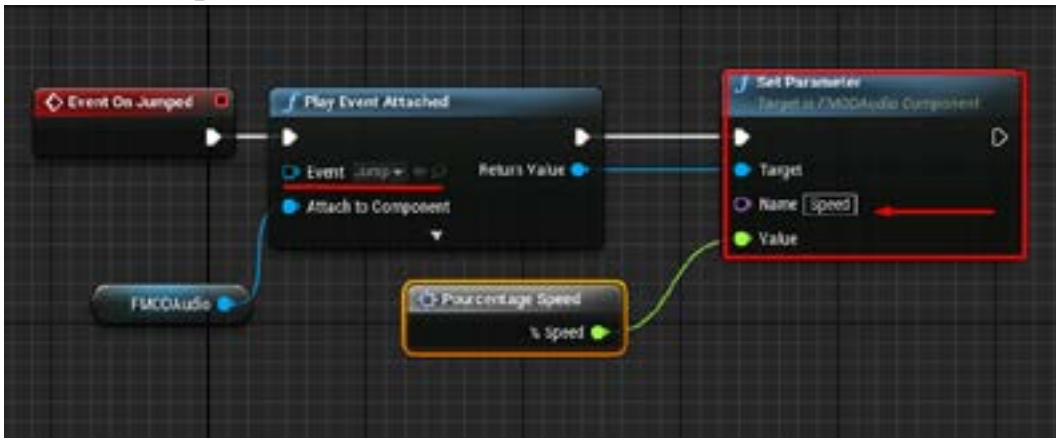
Création sonore

Pour l'intégration et changement du parameters de vitesse nous avons mis une macro réutilisable :



On prends la vélocité actuel du joueur divisé par la vélocité max en la multipliant par 100 pour donner une valeur entre 0 et 100.

Ainsi, par exemple, pour l'événement Jump, le son est joué a chaque fois que le joueur saute et en fonction du parameter vitesse qui est modifié en temps réel :



Création sonore

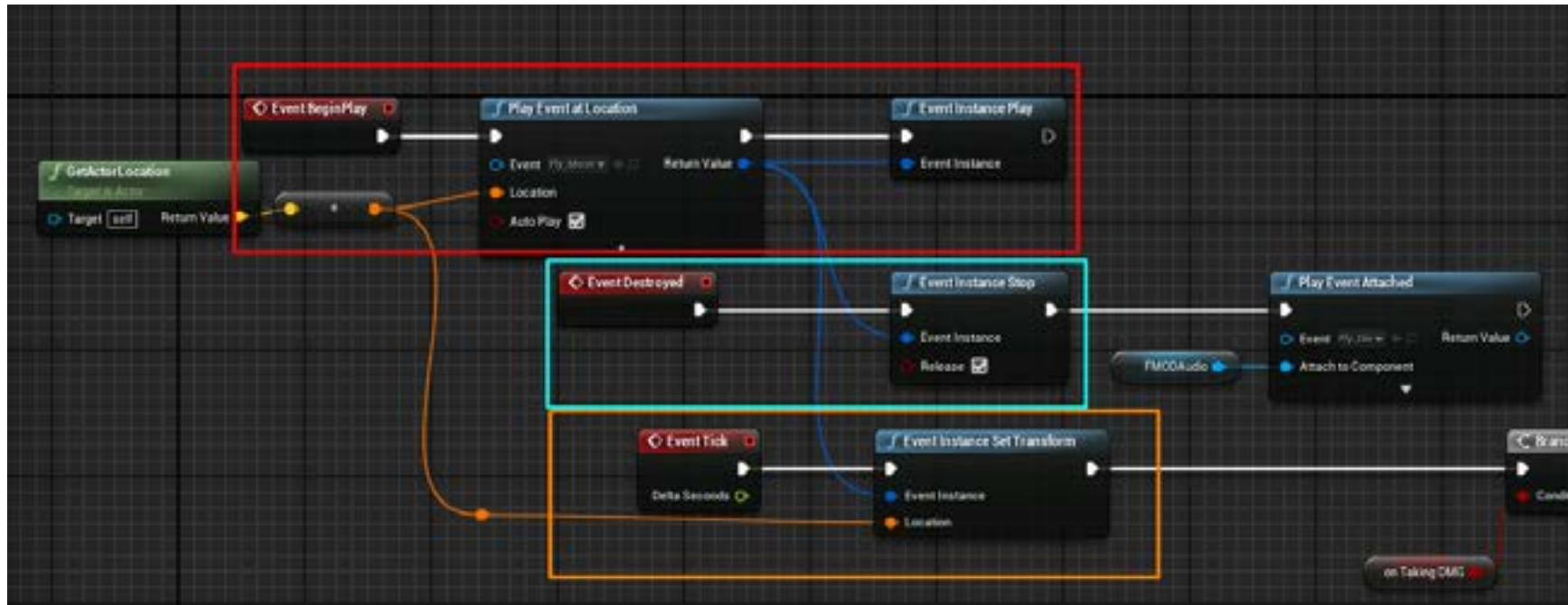
Pour les events contenant pas de loop, l'intégration est simple :



Ici comme exemple pour le tir, si en condition je peux tirer, alors ca joue le son attaché à l'objet.

Création sonore

Cependant, les event contenant des loops c’est plus compliqué et a été un réel casse tête :



Ici pour exemple le déplacement du FlyMob, la partie en rouge permet de jouer l’évent et le met en instance qui lié avec la partie en orage permettant de garder le position du son sur le FlyMob. Ainsi, la partie bleue permet d'arrêter l’évent une fois que le FlyMob est détruit.

Boucles d’objectifs
Court

Objectifs	Challenges	Rewards
Toucher une unité adverse	Viser un ennemi	Faire des dégâts à un ennemi
Détruire un ennemi	Réduire le nombre de PV de l’unité adverse à 0	Un ennemi en moins dans l’espace de jeu
Utiliser les tirs d’une arme	Avoir suffisamment de tir disponible dans le chargeur	Feedback de l’arme

Boucles d’objectifs

Moyen terme

Objectif	Challenge	Reward
Terminer une map	Réduire durant la partie le nombre de mobs existant dans la map à 0 Gérer les armes de l’avatar - Gestion des tirs - Bonne usage du rechargements - Utilisation des buffs d’armes	Accès à la map suivante.

Boucles d’objectifs

Long terme

Objectif	Challenges	Reward
Résister aux défis de la map actuelle	- Optimisation de l’arsenal (armes, terrain,...) - Planification et stratégie pour prendre l’avantage. Gérer les éléments du terrain de jeu - Maitrise des armes - Gestion des tirs - Utilisation du bunny hop - Maitrise des buffs lié aux armes	Satisfaction du Joueur liée à la validation de la map Débloque l’accès à la prochaine map

Améliorations

Dans le futur, il y a différents points que nous aimerions pousser/améliorer :

- Affiner les animations sur les mobs*
- Affiner les VFX*
- Ajouter des animations idle*
- IA plus intelligente*
- Ajouts de son manquants*